

09 - Postman API Testing

Import two files, click Run, and test all 25 endpoints in a few minutes.

What is this?

Postman is a free desktop app for sending API requests by hand and checking what comes back. It replaces the frontend entirely.

Like: phoning the godown directly instead of walking to the counter. If the counter is broken you still want to know whether the godown is answering.

WHY

Why bother when there is a frontend

When something is wrong, you need to know which of the three pieces broke. If Postman gets the right answer but the screen shows an error, the fault is in the frontend. If Postman also fails, the fault is in the backend or the database. This is how you avoid guessing.

The two files

Both live in the repository at:

```
docs/module-01-authentication-user-management/postman/  
    hardware-erp-module-01.postman_collection.json  
    hardware-erp-module-01.postman_environment.json
```

File	What it holds
collection	54 requests in 6 folders, with automatic checks on each
environment	The base URL and the four seed logins

IMPORTANT

These are generated, not hand-written

Both files are produced by `generate_postman.py` in the same folder, which reads the real controllers. Every request matches an endpoint that genuinely exists. If you add an endpoint, edit that script and re-run it rather than editing the JSON by hand.

Before you start

SETUP

Three things must be running or installed

1. **PostgreSQL is running.** From the project root: `docker compose up -d`
2. **The backend is running.** `cd backend && ./mvnw spring-boot:run`
3. **Postman is installed.** Download from postman.com/downloads. The free account is enough; you can also skip the account and use it offline.

Check the backend is up before opening Postman:

RUN THIS

Confirm the backend is answering

```
curl http://localhost:8080/api/actuator/health
```

WHAT YOU SHOULD SEE

Expected output

```
{"status": "UP"}
```

If you get "connection refused", the backend is not running. Go to document 17.

Step 1 - Import the collection

1. Open Postman
2. Click **Import** (top left, next to the workspace name)
3. Drag both JSON files onto the drop area, or click **files** and select them
4. Click **Import**

WHAT YOU SHOULD SEE

What you should see

In the left sidebar under **Collections**, a new entry:

"Hardware ERP - Module 1 - Authentication & User Management" with 6 folders inside it.

Step 2 - Select the environment

This is the step people forget, and every request fails without it.

1. Look at the **top right** of the Postman window
2. There is a dropdown that says **No Environment**
3. Change it to **Hardware ERP - Local (PostgreSQL)**

IMPORTANT

If you skip this

Every request will fail with an error about `{{baseUrl}}` being undefined, because Postman has no value for it. The dropdown is easy to miss.

The environment holds these values:

Variable	Value	Meaning
baseUrl	http://localhost:8080/api	Where the backend is listening
ownerIdentifier	9876543210	The owner's mobile number
ownerPassword	Owner@2026	The owner's password
managerIdentifier	9840112233	The manager's mobile number
staffIdentifier	9843012345	Counter staff mobile number
swaggerUrl	http://localhost:8080/api/swagger-ui.html	The Swagger page

IMPORTANT

These passwords are development-only
 They exist only in db/seed/V900__seed_dev_data.sql, which the production Flyway configuration deliberately does not load. A real deployment starts with exactly one account, created from environment variables you choose.

Step 3 - Log in

1. Open folder **1. Authentication**
2. Click **Login as OWNER**
3. Click the blue **Send** button

WHAT YOU SHOULD SEE

What you should see

Status **200 OK** in the response bar, and a body like this:

```
{
  "success": true,
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiJ9...",
    "refreshToken": null,
    "tokenType": "Bearer",
    "expiresInSeconds": 900,
    "mustChangePassword": false,
    "user": { "fullName": "Saravanan Murugan", "roleCode": "OWNER", ... }
  },
  "timestamp": "2026-08-14T09:14:22.331+05:30"
}
```

At the bottom of the response pane, a **Test Results** tab shows 6 passing checks in green.

You do **not** have to copy the token anywhere. A script attached to this request saves it automatically into a collection variable called `accessToken`, and every other request already sends `Authorization: Bearer {{accessToken}}`.

WHY

Why refreshToken is null

The refresh token is deliberately **not** in the response body. The backend sends it as an HttpOnly cookie, which JavaScript cannot read - so a cross-site-scripting bug in the frontend cannot steal it. Postman's cookie jar stores it and replays it automatically, which is why the ****Refresh token pair**** request works with an empty body.

Step 4 - Test a protected endpoint

Still in folder 1, click **Current user (GET /auth/me)** and Send.

WHAT YOU SHOULD SEE

Expected

Status **200**, and a body listing your name, role and the full permission array. The Test Results tab confirms the response contains no password hash.

Now prove the protection is real. Open folder **5. Security Tests** and run **401 - no token**.

WHAT YOU SHOULD SEE

Expected

Status **401 Unauthorized**, code UNAUTHENTICATED. The same endpoint that just worked now refuses you, because that request deliberately sends no token.

Step 5 - Run everything at once

This is the fast path.

1. Hover over the collection name in the sidebar
2. Click the ... menu, then **Run collection**
3. Leave every request ticked, in the given order
4. Click **Run Hardware ERP - Module 1**

Postman runs all 54 requests in order and shows a pass/fail summary.

IMPORTANT

Folder 5 asserts failures on purpose

Every request in **5. Security Tests** expects to be rejected. A 401 or 403 there is a **pass**. If one of those requests returns 200, something is genuinely wrong with the security configuration.

WHAT YOU SHOULD SEE

What a healthy run looks like

Folders 1 to 4 and 6 all green. Folder 5 all green, because each of its requests asserted the rejection it expected. Two requests may legitimately vary:

- **Revoke one session** passes with either 204 or 404
- **Change my password** is expected to be run manually, not in a full run

What each folder covers

1. Authentication (15 requests)

Login as owner, manager and staff. Login by email instead of mobile. Current user. Update profile. Refresh. Sessions list and revoke. Forgot password, twice

- once for a real account and once for an account that does not exist, and the collection asserts the two responses are **byte-identical**. Reset password.

Change password. Logout, and logout-all.

2. Users (11 requests)

List, search, filter by status, and the two safety checks: page size clamped to 100, and an SQL-injection attempt in the `sortBy` parameter that must fall back to the default instead of reaching the database. Then full create, read, update, admin password reset, delete, and delete-again-expecting-404.

3. Roles and Permissions (7 requests)

List roles and confirm STAFF genuinely lacks `PRODUCT_VIEW_COST`. Create, update and delete a custom role. List all permissions and the module-grouped form used by the role screen.

4. Security Audit Log (3 requests)

Search all events, filter to failed logins, and filter to refresh-token reuse detection. Requires `AUDIT_VIEW`, which only OWNER holds.

5. Security Tests (16 requests)

Request	Expected	Why it matters
No token	401	The endpoint is genuinely protected
Malformed JWT	401	Rubbish input does not crash the parser
Tampered signature	401	Editing the token invalidates it

Request	Expected	Why it matters
Wrong password	401	Rejected, with a generic message
Unknown account	401, identical body	Login cannot be used to discover accounts
Inactive account	401	Same generic response
Soft-deleted account	401	Deleted users cannot sign in
STAFF lists users	403	Permission enforced server-side
MANAGER creates a role	403	Permission enforced server-side
Bad field values	400 with field names	Backend validates, not just the UI
Duplicate mobile	409	Unique constraint holds
Duplicate email, different case	409	Case-insensitive uniqueness holds
OWNER loses a permission	422	The shop cannot lock itself out
Delete a system role	422	System roles are protected
Unknown user id	404	Correct status, not 500
Any request	X-Request-ID present	Support can trace it in the logs

WHY

Why the enumeration check matters most

"401 - unknown account is indistinguishable" compares the exact response body against the wrong-password response, ignoring only timestamp and request id.

If they ever differ, an attacker can feed in mobile numbers and learn which ones are registered - a list of real accounts to attack. This test is the guard on that.

6. Rate Limiting (2 requests)

IMPORTANT

The dev profile relaxes rate limits on purpose

application-dev.yml allows 100 logins per minute so that manual testing is not throttled. To see a real 429 you must start the backend with the default profile:

```
SPRING_PROFILES_ACTIVE=default ./mvnw spring-boot:run
```

That gives 10 logins per minute per IP and 3 forgot-password requests per hour. Then use the Runner on **Hammer login until 429** with 15 iterations.

The full path of one request

POSTMAN

```
| POST /api/v1/auth/login { identifier, password }
v
```

SPRING BOOT (port 8080, context path /api)

```

|
v
RateLimitFilter      too many attempts from this IP?  -> 429
|
v
JwtAuthenticationFilter  skipped: /auth/login is public
|
v
AuthController         validates the request body      -> 400
|
v
AuthServiceImpl         account exists? locked? active?
                        password matches the BCrypt hash? -> 401
|
v
UserRepository          SELECT ... FROM app_user WHERE ...
|
v
POSTGRES               reads the row
|
v
... back up, and a JWT plus a Set-Cookie header return to Postman

```

If it fails

IF IT FAILS

"Error: connect ECONNREFUSED 127.0.0.1:8080"

The backend is not running. Start it with `./mvnw spring-boot:run` from the backend folder and wait for the "Started HardwareErpApplication" line.

IF IT FAILS

Every request shows the URL as "{{baseUrl}}/v1/..."

You have not selected the environment. Use the dropdown at the top right and pick "Hardware ERP - Local (PostgreSQL)".

IF IT FAILS

Login returns 401 with the seed password

The seed data has not loaded. It only loads under the dev or test profile.

Confirm the backend started with the dev profile - the console prints

The following 1 profile is active: "dev". Then check the data is really there:

```

docker exec -it hardware-erp-postgres psql -U hardware_erp -d hardware_erp -c "SELECT mobile_no, full_name FROM app_user LIMIT 5;"

```

IF IT FAILS

Everything returns 401 after about 15 minutes

The access token has expired - that is correct behaviour. Run ****Login as OWNER again, or run Refresh token pair****, which uses the cookie.

IF IT FAILS

"Refresh token pair" returns 401 TOKEN_REUSE

You have replayed a refresh token that was already rotated. The backend treats that as theft and revokes every session for the user - which is the intended behaviour. Simply run **Login as OWNER** again to start a fresh session.

IF IT FAILS

Requests in folder 2 fail with 404 on {{createdUserId}}

You ran the update or delete request without running **Create user** first. That request is what sets the variable. Run the folder in order.

After: how to know you are done

VERIFY

Module 1 API testing is complete when

1. The Collection Runner reports every request green
 2. Folder 5 is green, meaning every rejection happened as expected
 3. GET /v1/auth/me returns your own name and role
 4. GET /v1/users returns the seeded users and excludes the soft-deleted one
 5. The security audit log shows the LOGIN_SUCCESS events your run created
- Point 5 is the real proof: it shows the request reached the service layer, committed a transaction, and wrote to PostgreSQL.